

014: StepPPO-v3 方法说明: GiGPO 上的 Shared Value Head

2026-06-09

摘要

当前分支中的 StepPPO-v3 是一个刻意保持最小改动的 value-head-only 版本。它不改变 GiGPO 的 policy advantage、policy loss、rollout group 语义或训练 recipe; 唯一新增项是在 actor backbone 上挂一个 shared value head, 并用 step-level GAE return 训练这个 head。v3 的研究问题不是提出新的 policy-gradient estimator, 而是检验 StepPPO-v2 中使用的 actor-side value head 是否能够稳定学习 step value, 并且不会干扰 GiGPO 原有训练行为。

1. 问题定义

考虑一个 WebShop multi-turn trajectory:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}).$$

其中, s_t 表示第 t 次环境交互前的文本上下文, a_t 表示语言模型生成并由环境执行的完整 textual action, r_t 表示该 step 的标量 reward。实现中, 每个 step sample 通过 `traj_uid` 与 `step_idx` 确定其所在 trajectory 及时间顺序。

GiGPO baseline 使用其原有 group-relative advantage, 记为 $\hat{A}_i^{\text{GiGPO}}$, 优化 actor policy π_θ 。StepPPO-v3 不改变这个 policy advantage, 而是在同一个 actor 上增加 value head v_ψ , 为每个 environment step 预测一个 scalar state value:

$$V_\psi(s_i) \in \mathbb{R}.$$

2. v3 的方法边界

v3 的核心问题只有两个:

1. shared value head 能否稳定拟合 step-level value/return;
2. 加入 value head 后, GiGPO 的 policy 训练是否保持不退化。

因此, 当前 v3 默认不包含以下内容:

- 新的 `step_ppo_v3` advantage estimator;
- pure step advantage 替代 GiGPO advantage;
- action-level 或 sequence-level PPO ratio;
- separate critic;
- 额外复杂 reward shaping。

换言之, v3 是 GiGPO 上的 shared value head 实验, 而不是一个新的 policy algorithm。

3. Shared Value Head 参数化

令 x_i 表示第 i 个 step sample 的 tokenized context-response sequence, actor backbone 最后一层 hidden states 为:

$$H_i = f_\theta(x_i) \in \mathbb{R}^{L_i \times d}.$$

当前实现默认在 action 前最后一个 context token 位置取 value:

$$b_i = L_i - |a_i| - 1.$$

主线 v3 使用 detached value backbone:

$$z_i = \text{sg}(H_i[b_i]), \quad V_\psi(s_i) = v_\psi(z_i),$$

其中 $\text{sg}(\cdot)$ 是 stop-gradient operator。该设定对应:

```
actor_rollout_ref.actor.value_head.enable=True
actor_rollout_ref.actor.value_head.value_position=pre_action_last_context_token
actor_rollout_ref.actor.value_head.detach_value_backbone=True
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.value_head.cliprange_value=0.5
```

因此, value loss 只更新 ψ , 不通过 value regression 更新 actor backbone θ 。GiGPO policy gradient 仍按原路径更新 θ 。

4. Step-Level Value Target

value head 的 target 来自 step-level GAE return, 而不是来自 GiGPO 的 policy advantage。对 step i , 先构造 value target 使用的 immediate reward:

$$\tilde{r}_i = r_i - \alpha \mathbf{1}\{\text{invalid}_i\} + \Delta r_i^{\text{KL}},$$

其中 α 是 `invalid_action_penalty_coef`。当 `algorithm.use_kl_in_reward=False` 时, $\Delta r_i^{\text{KL}} = 0$, 这也是当前 WebShop v3 脚本的默认行为。

给定 old value prediction V_i^{old} , 在同一 `traj_uid` 内按 `step_idx` 反向计算:

$$\delta_t = \tilde{r}_t + \gamma(1 - d_t)V_{t+1}^{\text{old}} - V_t^{\text{old}},$$

$$\hat{A}_t^V = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}^V,$$

并得到 scalar value target:

$$\hat{R}_t^V = \hat{A}_t^V + V_t^{\text{old}}.$$

当前主线配置为:

```
algorithm.gigpo_v1_value_aux.value_target=gae_returns
algorithm.gigpo_v1_value_aux.step_gamma=0.95
algorithm.gigpo_v1_value_aux.step_lam=0.90
```

实现中, $\hat{R}_t^V$ 写入 `data.batch["step_returns"]`; value target 使用的 shaped reward 写入 `data.batch["value_step_rewards"]`, 用于诊断 reward 口径。

5. 优化目标

v3 的总 loss 是 GiGPO policy loss 加 auxiliary value loss:

$$\mathcal{L}_{v3}(\theta, \psi) = \mathcal{L}_{\text{GiGPO}}(\theta; \hat{A}^{\text{GiGPO}}) + c_v \mathcal{L}_V(\psi).$$

这里 $c_v = 0.03$ 。注意 $\hat{A}^{\text{GiGPO}}$ 不由 value head 产生, policy 更新不读取 $\hat{A}^V$ 。

value loss 使用 clipped value objective。对 minibatch $\mathcal{B}$, 定义:

$$V_i^{\text{clip}} = V_i^{\text{old}} + \text{clip}(V_\psi(s_i) - V_i^{\text{old}}, -\epsilon_v, \epsilon_v).$$

则:

$$\mathcal{L}_V = \frac{1}{2|\mathcal{B}|} \sum_{i \in \mathcal{B}} \max \left[\left(V_\psi(s_i) - \hat{R}_i^V \right)^2, \left(V_i^{\text{clip}} - \hat{R}_i^V \right)^2 \right].$$

在 `detach_value_backbone=True` 下, $\nabla_\theta \mathcal{L}_V = 0$, 从而避免 value regression 直接扰动 GiGPO actor backbone。

6. 诊断指标与通过标准

只观察 `actor/value_loss` 不足以判断 value head 是否学到有效 signal。当前实现应记录:

```
actor/value_loss
actor/value_clipfrac
actor/value_rmse
actor/value_mae
actor/value_explained_variance
actor/value_pred_mean
actor/value_pred_std
actor/value_return_mean
actor/value_return_std
```

其中 explained variance 定义为:

$$\text{EV} = 1 - \frac{\text{Var}(\hat{R}^V - V_\psi)}{\text{Var}(\hat{R}^V) + \varepsilon}.$$

v3 的最低通过标准是: value RMSE/MAE 稳定, EV 出现正向 signal, 并且以下 GiGPO 行为指标相对 no-value-head baseline 不明显退化:

```
episode/valid_action_ratio
response_length/mean
response_length/clip_ratio
actor/pg_loss
actor/grad_norm
val/success_rate
val/webshop_task_score (not success_rate)
```

7. 主实验脚本

主线脚本为:

```
exps/run_webshop_step_ppo_v3_value_head_4gpu_paper_align_simple.sh
```

其核心 overrides 为:

```
algorithm.adv_estimator=gigpo
algorithm.gigpo.step_advantage_w=1.0
algorithm.gigpo.mode=mean_norm
algorithm.gigpo_v1_value_aux.value_target=gae_returns
algorithm.gigpo_v1_value_aux.step_gamma=0.95
algorithm.gigpo_v1_value_aux.step_lam=0.90
actor_rollout_ref.actor.value_head.enable=True
actor_rollout_ref.actor.value_head.detach_value_backbone=True
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.value_head.cliprange_value=0.5
```

脚本名使用 StepPPO-v3 作为研究标签; 方法含义是 GiGPO + detached shared value head。

8. 与当前代码实现的一致性检查

当前代码与上述方法一致, 逐项对应如下:

1. **GiGPO policy 不变**。GiGPOV1ValueAuxRayTrainer 要求 `algorithm.adv_estimator=gigpo`, 并先调用原始 `compute_advantage`, 再附加 value target。这与 v3 不改变 policy advantage 一致。
2. **value target 来自 step GAE**。 `_attach_gae_value_targets` 调用 `compute_step_gae_advantage_return`, 并写入 `data.batch["step_returns"]`, 对应 $\hat{R}^V$ 。
3. **shaped immediate reward**。 `_build_shaped_immediate_rewards` 读取 raw rewards, 扣 invalid-action penalty, 并在需要时加入 KL reward delta, 对应 $\tilde{r}_i$ 。
4. **pre-action value position**。 `StepWisePPOActor._select_state_values` 在 `pre_action_last_context_token` 下使用 `-response_length-1`, 对应 $b_i = L_i - |a_i| - 1$ 。
5. **detach backbone**。 `value_head.py` 在 `detach_value_backbone=True` 时对 hidden states 执行 detach, 对应 $z_i = \text{sg}(H_i[b_i])$ 。
6. **auxiliary value loss**。 `StepWisePPOActor.update_policy` 仅在 `step_returns` 存在时加入 `value_loss_coef*value_loss`, 对应总 loss 中的 $c_v \mathcal{L}_V$ 。
7. **无 action-level PPO**。当前 actor 仍只 dispatch 到已有 vanilla 或 gspo policy loss; v3 脚本没有设置 action-level loss。这与 v3 方法边界一致。
8. **rollout sync 安全**。 `step_ppo_filters.py` 会从 rollout state dict 中过滤 `value_head.*`, 避免 vLLM/sglang 加载 auxiliary value head 参数。

唯一需要说明的是命名: 实现复用了 GiGPO-v1 value-aux trainer 和 StepPPO 的 value-head worker class。该复用不构成方法不一致, 因为当前 v3 本身就是 value-head-only 的 GiGPO 变体, 而不是新的 estimator 实现。